

trivial union (was `std::uninitialized<T>`)

Document #: P3074R3
Date: 2024-04-14
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
 - [2.1 The uninitialized storage problem](#)
- [3 Design Space](#)
 - [3.1 A library_type: `std::uninitialized<T>`](#)
 - [3.2 A language annotation](#)
 - [3.3 Just make it work](#)
 - [3.4 Trivial Unions](#)
 - [3.5 Existing Practice](#)
- [4 Proposal](#)
 - [4.1 Wording for just making it work](#)
 - [4.2 Wording for trivial union](#)
 - [4.3 Feature-Test Macro](#)
- [5 References](#)

§ 1 Revision History

[[P3074R0](#)] originally proposed the function `std::start_lifetime(p)`. This revision adds a new section discussing the [uninitialized storage problem](#), which motivates a change in design to instead propose `std::uninitialized<T>`.

[[P3074R1](#)] changed to propose `std::uninitialized<T>` and was discussed in an EWG telecon. There, the suggestion was made to make this a language feature, which this revision discusses and argues against. Also re-spelled `std::uninitialized<T>` to be a union instead of a class containing an anonymous union.

[[P3074R2](#)] still argued for `std::uninitialized<T>`. This revision changes to instead proposing a language change to unions to solve the problems presented.

§ 2 Introduction

Consider the following example:

```

template <typename T, size_t N>
struct FixedVector {
    union U { constexpr U() { } constexpr ~U() { } T storage[N]; };
    U u;
    size_t size = 0;

    // note: we are *not* constructing storage
    constexpr FixedVector() = default;

    constexpr ~FixedVector() {
        std::destroy(u.storage, u.storage+size);
    }

    constexpr auto push_back(T const& v) -> void {
        std::construct_at(u.storage + size, v);
        ++size;
    }
};

constexpr auto silly_test() -> size_t {
    FixedVector<std::string, 3> v;
    v.push_back("some sufficiently longer string");
    return v.size;
}

static_assert(silly_test() == 1);

```

This is basically how any static/non-allocating/in-place vector is implemented: we have some storage, that we *definitely do not value initialize* and then we steadily construct elements into it.

The problem is that the above does not work (although there is [implementation divergence](#) - MSVC and EDG accept it and GCC did accept it even up to 13.2, but GCC trunk and Clang reject).

Getting this example to work would allow `std::inplace_vector` ([[P0843R9](#)]) to simply work during `constexpr` time for all times (instead of just trivial ones), and was a problem briefly touched on in [[P2747R0](#)].

§ 2.1 The uninitialized storage problem

A closely related problem to the above is: how do you do uninitialized storage? The straightforward implementation would be to do:

```

template <class T>
struct BufferStorage {
private:
    alignas(T) unsigned char buffer[sizeof(T)];

public:
    // accessors
};

```

This approach generally works, but it has two limitations:

1. it cannot work in `constexpr` and that's likely a fundamental limitation that will never change, and
2. it does not quite handle overlapping objects correctly.

What I mean by the second one is basically given this structure:

```
struct Empty { };

struct Sub : Empty {
    BufferStorage<Empty> buffer_storage;
};
```

If we initialize the `Empty` that `buffer_storage` is intended to have, then `Sub` has two subobjects of type `Empty`. But the compiler doesn't really... know that, and doesn't adjust them accordingly. As a result, the `Empty` base class subobject and the `Empty` initialized in `buffer_storage` are at the same address, which violates the rule that all objects of one type are at unique addresses.

An alternative approach to storage is to use a `union`:

```
template <class T>
struct UnionStorage {
private:
    union { T value; };

public:
    // accessors
};

struct Sub : Empty {
    UnionStorage<Empty> union_storage;
};
```

Here, now the compiler knows for sure there is an `Empty` in `union_storage` and will lay out the types appropriately. See also [gcc bug 112591](#).

So it seems that the `UnionStorage` approach is strictly superior: it will work in `constexpr` and it lays out overlapping types properly. But it has limitations of its own. As with the `FixedVector` example earlier, you cannot just start the lifetime of `value`. But also in this case we run into the `union` rules for special member functions: a special member of a `union`, by default, is either trivial (if that special member for all alternatives is trivial) or deleted (otherwise). Which means that `UnionStorage<std::string>` has both its constructor and destructor *deleted*.

We can work around this by simply adding an empty constructor and destructor (as shown earlier as well):

```

template <class T>
struct UnionStorage2 {
private:
    union U { U() { } ~U() { } T value; };
    U u;

public:
    // accessors
};

```

This is a fundamentally weird concept since U there has a destructor that does nothing (and given that this is a class to be used for uninitialized storage), it *should* do nothing - that's correct. But that destructor still isn't trivial. And it turns out there is still a difference between “destructor that does nothing” and “trivial destructor”:

Trivially Destructible	Non-trivially Destructible
<pre> struct A { }; auto alloc_a(int n) -> A* { return new A[n]; } auto del(A* p) -> void { delete [] p; } </pre>	<pre> struct B { ~B() { } }; auto alloc_b(int n) -> B* { return new B[n]; } auto del(B* p) -> void { delete [] p; } </pre>
<pre> alloc_a(int): movsx rdi, edi jmp operator new[] (unsigned long) del(A*): test rdi, rdi je .L3 jmp operator delete[] (void*) .L3: ret </pre>	<pre> alloc_b(int): movabs rax, 9223372036854775800 push rbx movsx rbx, edi cmp rax, rbx lea rdi, [rbx+8] mov rax, -1 cmovb rdi, rax call operator new[] (unsigned long) mov QWORD PTR [rax], rbx add rax, 8 pop rbx ret del(B*): test rdi, rdi je .L9 mov rax, QWORD PTR [rdi-8] sub rdi, 8 lea rsi, [rax+8] jmp operator delete[] (void*, unsigned long) .L9: ret </pre>

That's a big difference in code-gen, due to the need to put a cookie in the allocation so that the corresponding `delete[]` knows how many elements there so that their destructors (even though they do nothing!) can be invoked.

While the union storage solution solves some language problems for us, the buffer storage solution can lead to more efficient code - because `StorageBuffer<T>` is always trivially destructible. It would be nice if he had a good solution to all of these problems - and that solution was also the most efficient one.

§ 3 Design Space

There are several potential solutions in this space:

1. a library solution (add a `std::uninitialized<T>`)
2. a language solution (add some annotation to members to mark them uninitialized, as distinct from `unions`)
3. just make it work (change the union rules to implicitly start the lifetime of the first alternative, if it's an implicit-lifetime type)
4. introduce a new kind of union
5. provide an explicit function to start lifetime of a union alternative (`std::start_lifetime`).

The first revision of this paper ([P3074R0]) proposed that last option. However, with the addition of the overlapping subobjects problem and the realization that the union solution has overhead compared to the buffer storage solution, it would be more desirable to solve both problems in one go. That is, it's not enough to just start the lifetime of the alternative, we also want a trivially constructible/destructible solution for uninitialized storage.

[P3074R1] and [P3074R2] proposed the first solution (`std::uninitialized<T>`). This revision proposes the third or fourth.

Let's go over some of the solutions.

§ 3.1 A library type: `std::uninitialized<T>`

We could introduce another magic library type, `std::uninitialized<T>`, with an interface like:

```
template <typename T>
struct uninitialized {
    union { T value; };
};
```

As basically a better version of `std::aligned_storage`. Here is storage for a `T`, that implicitly begins its lifetime if `T` is an implicit-lifetime-type, but otherwise will not actually initialize it for you - you have to do that yourself. Likewise it will not destroy it for you, you have to do that yourself too. This type would

be specified to always be trivially default constructible and trivially destructible. It would be trivially copyable if T is trivially copyable, otherwise not copyable.

`std::inplace_vector<T, N>` would then have a `std::uninitialized<T[N]>` and go ahead and `std::construct_at` (or, with [P2747R2], simply placement-new) into the appropriate elements of that array and everything would just work.

Because the language would recognize this type, this would also solve the overlapping objects problem.

§ 3.2 A language annotation

During the EWG telecon in [January 2023](#), the suggestion was made that instead of a magic library type like `std::uninitialized<T>`, we could instead have some kind of language annotation to achieve the same effect.

For example:

```
template <typename T, size_t N>
struct FixedVector {
    // as a library feature
    std::uninitialized<T[N]> lib;

    //as a language feature, something like this
    for storage T lang[N];
    T storage[N] = for lang;
    T storage[N] = void;
    uninitialized T lang[N];

    size_t size = 0;
};
```

The advantage of the language syntax is that you can directly use `lang` - you would placement new onto `lang[0]`, you read from `lang[1]`, etc, whereas with the library syntax you have to placement new onto `lib.value[0]` and read from `lib.value[1]`, etc.

In that telecon, there was preference (including by me) for the language solution:

SF	F	N	A	SA
5	4	4	2	1

However, an uninitialized object of type T really isn't the same thing as a T. `decltype(lang)` would have to be T, any kind of (imminent) reflection over this type would give you a T. But there might not actually be a T there yet, it behaves like a `union { T; }` rather than a T, so spelling it T strikes me as misleading.

We would have to ensure that all the other member-wise algorithms we have today (the special member functions and the comparisons) use the “uninitialized T” meaning rather than the T meaning. And with reflection, that also means all future member-wise algorithms would have to account for this also - rather than rejecting unions. This seems to open the door to a lot of mistakes.

The syntactic benefits of the language syntax are nice, but this is a rarely used type for specific situations - so having slightly longer syntax (and really, `lib.value` is not especially cumbersome) is not only not a big downside here but could even be viewed as a benefit.

For this reason, R2 of this paper still proposed `std::uninitialized<T>` as the solution in preference to any language annotation. This did not go over well in [Tokyo](#), where again there was preference for the language solution:

SF	F	N	A	SA
6	7	3	4	2

This leads to...

§ 3.3 Just make it work

Now, for the `inplace_vector` problem, today's `union` is insufficient:

```
template <typename T, size_t N>
struct FixedVector {
    union { T storage[N]; };
    size_t size = 0;
};
```

Similarly a simple `union { T storage; }` is insufficient for the uninitialized storage problem.

There are three reasons for this:

1. the default constructor can be deleted (this can be easily worked around though)
2. the default constructor does not start the lifetime of implicit lifetime types
3. the destructor can be deleted (this can be worked around by providing a no-op destructor, which has ABI cost that cannot be worked around)

However, what if instead of coming up with a solution for these problems, we just... made it work?

That is, change the union rules as follows:

member	status quo	new rule
default constructor (absent default member initializers)	If all the alternatives are trivially default constructible, trivial. Otherwise, deleted.	If the first alternative is an implicit-lifetime type, trivial and starts the lifetime of that alternative and sets it as the active member. Otherwise, if all the alternatives are trivially

member	status quo	new rule
		default constructible, trivial. Otherwise, deleted.
destructor	If all the alternatives are trivially destructible, trivial. Otherwise, deleted.	If the first alternative is an implicit-lifetime type or if all the alternatives are trivially default constructible, trivial. Otherwise, deleted.

This attempt at a minimal extension works fine for the `inplace_vector` example where we want a union holding a `T[N]`. Such a union would become trivially default constructible (and start the lifetime of the array) and trivially destructible, as desired. But it has odd effects for the typical uninitialized storage case:

```
// default constructor and destructor are both deleted
union U1 { std::string s; };

// default constructor and destructor are both trivial
union U2 { std::string a[1]; };
```

For uninitialized storage, we really want trivial construction/destruction. And it would be nice to not have to resort to having members of type `T[1]` instead of `T` to achieve this. But I really don't think it's a good idea to just make all unions trivially constructible and destructible. Seems a bit too late for that. However...

§ 3.4 Trivial Unions

What if we introduced a new kind of union, with special annotation? That is:

```
template <typename T, size_t N>
struct FixedVector {
    trivial union { T storage[N]; };
    size_t size = 0;
};
```

With the rule that a trivial union is just always trivially default constructible, trivially destructible, and, if the first alternative is implicit-lifetime, starts the lifetime of that alternative (and sets it to be the active member).

This is a language solution that doesn't have any of the consequences for memberwise algorithms - since we're still a `union`. It provides a clean solution to the uninitialized storage problem, the aliasing problem, and the `constexpr inplace_vector` storage problem. Without having to deal with potentially changing behavior of existing unions.

This brings up the question about default member initializers. Should a `trivial union` be allowed to have a default member initializer? I don't think so. If you're initializing the thing, it's not really

uninitialized storage anymore. Use a regular union.

An alternative spelling for this might be uninitialized `union` instead of trivial `union`. An alternative alternative would be to instead provide a different way of declaring the constructor and destructor:

```
union U {  
    U() = trivial;  
    ~U() = trivial;  
  
    T storage[N];  
};
```

This is explicit (unlike [just making it work](#)), but seems unnecessary much to type compared to a single trivial token - and these things really aren't orthogonal. Plus it wouldn't allow for anonymous trivial unions, which seems like a nice usability gain.

§ 3.5 Existing Practice

There are three similar features in other languages that I'm aware of.

Rust has [MaybeUninit<T>](#) which is similar to what's described here as `std::uninitialized<T>`.

Kotlin has a [lateinit var](#) language feature, which is similar to some kind of language annotation (although additionally allows for checking whether it has been initialized, which the language feature would not provide).

D has the ability to initialize a variable to `void`, as in `int x = void`; This leaves `x` uninitialized. However, this feature only affects construction - not destruction. A member `T[N] storage = void`; would leave the array uninitialized, but would destroy the whole array in the destructor. So not really suitable for this particular purpose.

§ 4 Proposal

This paper now proposes support for a new kind of union: [trivial union](#) with the following rules:

- a `union` may be declared trivial (this is a context-sensitive keyword). If any declaration of a union `U` contains trivial, every declaration of `U` must do so.
- a trivial `union` shall not have any default member initializers.
- a trivial `union` is trivially default constructible. If the first alternative has implicit-lifetime type, this also begins the lifetime of that alternative and sets it as the active member.
- a trivial `union` is trivially destructible.

The syntax is trivial `union` instead of `union trivial` (which might be more consistent with the use of `final`) because the former allows an anonymous union declaration as `trivial union { T n; }`

whereas `union trivial { T n; }` is already a valid declaration today. Nor can you put the `trivial` even later - as in `union { T n; } trivial` since now that is declaring a variable.

A better syntax that wouldn't lead to conversations about context-sensitive keywords would be `union [[trivial]]`.

Another potential choice of word instead of `trivial` here would be `uninitialized`.

An alternative design would be to change all existing `unions` to have this behavior (except still allowing default member initializers). That is:

trivial union	just make it work
<pre>// trivial default constructor // does not start lifetime of s // trivial destructor trivial union U1 { string s; }; // deleted default constructor // deleted destructor union U2 { string s; }; // trivial default constructor // starts lifetime of s // trivial destructor trivial union U3 { string s[10]; }</pre>	<pre>// trivial default constructor // does not start lifetime of s // trivial destructor union U4 { string s; }; // non-trivial default constructor // deleted destructor union U5 { string s = "hello"; } // trivial default constructor // starts lifetime of s // trivial destructor union U6 { string s[10]; }</pre>

It's worth discussing both options. Unions already have very sharp edges, so perhaps this added protection of deleting the default constructor and destructor aren't really super useful - that's probably not the feature that really saves you.

Note that just making work will change some code from ill-formed to well-formed, whereas introducing a trivial `union` will not change the meaning of any existing code.

§ 4.1 **Wording for just making it work**

Change 11.4.5.2 [\[class.default.ctor\]](#)/2-3. [Editor's note: The third and fourth bullets can be removed because such cases become trivially default constructible too]

2	A defaulted default constructor for class X is defined as deleted if X is a non-union class and:
(2.1)	— any non-static data member with no default member initializer ([class.mem]) is of reference type,
(2.2)	— any non-variant non-static data member of const-qualified type (or possibly multi-dimensional array thereof) with no brace-or-equal-initializer is not const-default-constructible ([dcl.init]),

- (2.3) — ~~X is a union and all of its variant members are of const-qualified type (or possibly multi-dimensional array thereof),~~
- (2.4) — ~~X is a non-union class and all members of any anonymous union member are of const-qualified type (or possibly multi-dimensional array thereof),~~
- (2.5) — any potentially constructed subobject, except for a non-static data member with a brace-or-equal-initializer ~~or a variant member of a union where another non-static data member has a brace-or-equal-initializer~~, has class type M (or possibly multi-dimensional array thereof) and overload resolution ([over.match]) as applied to find M's corresponding constructor either does not result in a usable candidate ([over.match.general]) ~~or, in the case of a variant member, selects a non-trivial function,~~ or

3 A default constructor **for a class X** is *trivial* if it is not user-provided and if:

- (3.1) — ~~its class X~~ has no virtual functions ([class.virtual]) and no virtual base classes ([class.mi]), and
- (3.2) — no non-static data member of ~~its class X~~ has a default member initializer ([class.mem]), and
- (3.3) — all the direct base classes of ~~its class X~~ have trivial default constructors, and
- (3.4) — **either X is a union or** for all the non-static data members of ~~its class X~~ that are of class type (or array thereof), each such class has a trivial default constructor.

Otherwise, the default constructor is *non-trivial*. **If the default constructor of a union X is trivial and the first variant member of X has implicit-lifetime type ([basic.types.general]), the default constructor begins the lifetime of that member [Note 1: It becomes the active member of the union — end note].**

Change 11.4.7 [class.dtor]/7-8:

7 A defaulted destructor for a class X is defined as deleted if **X is a non-union class and:**

- (7.1) — any potentially constructed subobject has class type M (or possibly multi-dimensional array thereof) and M has a destructor that is deleted or is inaccessible from the defaulted destructor ~~or, in the case of a variant member, is non-trivial,~~
- (7.2) — or, for a virtual destructor, lookup of the non-array deallocation function results in an ambiguity or in a function that is deleted or inaccessible from the defaulted destructor.

8 A destructor **for a class X** is *trivial* if it is not user-provided and if:

- (8.1) — the destructor is not virtual,
- (8.2) — all of the direct base classes of ~~its class X~~ have trivial destructors, and

- (8.3) — either `X` is a union with no default member initializer or for all of the non-static data members of ~~its class~~ `X` that are of class type (or array thereof), each such class has a trivial destructor.

§ 4.2 Wording for `trivial union`

Add `trivial` to the identifiers with special meaning table in 5.10 [\[lex.name\]](#):

```
final
import
module
override
+ trivial
```

Change 11.1 [\[class.pre\]](#) to add the ability to declare a `union trivial`:

```
class-key:
    class
    struct
-   union
+   trivialopt union
```

Add to the end of 11.1 [\[class.pre\]](#):

- 8 A *class-key* shall only contain `trivial` when used in a *class-head*. If any declaration of a union `U` has a `trivial` specifier, then all declarations of `U` shall contain `trivial` [*Note 1*: This includes those declarations that use an *elaborated-type-specifier*, which cannot provide the `trivial` specifier. — *end note*].

Add to 11.5.1 [\[class.union.general\]](#)/1:

- 1 A *union* is a class defined with the *class-key* `union`. A *trivial union* is a union defined with the *class-key* `trivial union`. A trivial union shall not have a default member initializer.

Change 11.5.2 [\[class.union.anon\]](#)/1:

- 1 A union of the form
- ```
- union { member-specification } ;
+ trivialopt union { member-specification } ;
```

is called an *anonymous union* [...]

Change 11.4.5.2 [\[class.default.ctor\]](#)/2-3.

2 A defaulted default constructor for class *X* is defined as deleted if *X* is not a trivial union and:

(2.1) — [...]

3 A default constructor for a class *X* is *trivial* if it is not user-provided and if:

(3.1) — ~~its class~~ *X* has no virtual functions ([class.virtual]) and no virtual base classes ([class.mi]), and

(3.2) — no non-static data member of ~~its class~~ *X* has a default member initializer ([class.mem]), and

(3.3) — all the direct base classes of ~~its class~~ *X* have trivial default constructors, and

(3.4) — either *X* is a trivial union or for all the non-static data members of ~~its class~~ *X* that are of class type (or array thereof), each such class has a trivial default constructor.

Otherwise, the default constructor is *non-trivial*. If the default constructor of a trivial union *X* is trivial and the first variant member of *X* has implicit-lifetime type ([basic.types.general]), the default constructor begins the lifetime of that member [ *Note 2: It becomes the active member of the union — end note* ].

Change 11.4.7 [\[class.dtor\]](#)/7-8:

7 A defaulted destructor for a class *X* is defined as deleted if *X* is not a trivial union and:

(7.1) — [...]

8 A destructor for a class *X* is *trivial* if it is not user-provided and if:

(8.1) — the destructor is not virtual,

(8.2) — all of the direct base classes of ~~its class~~ *X* have trivial destructors, and

(8.3) — either *X* is a trivial union or for all of the non-static data members of ~~its class~~ *X* that are of class type (or array thereof), each such class has a trivial destructor.

## § 4.3 Feature-Test Macro

Either way, we need a new feature-test macro. Add a new macro to 15.11 [\[cpp.predefined\]](#):

```
__cpp_trivial_union 2024XXL
```

## § 5 References

---

[P0843R9] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2023-09-14. `inplace_vector`.

<https://wg21.link/p0843r9>

[P2747R0] Barry Revzin. 2022-12-17. Limited support for `constexpr void*`.

<https://wg21.link/p2747r0>

[P2747R2] Barry Revzin. 2024-03-19. `constexpr` placement new.

<https://wg21.link/p2747r2>

[P3074R0] Barry Revzin. 2023-12-15. `constexpr` union lifetime.

<https://wg21.link/p3074r0>

[P3074R1] Barry Revzin. 2024-01-30. `std::uninitialized<T>`.

<https://wg21.link/p3074r1>

[P3074R2] Barry Revzin. 2024-02-13. `std::uninitialized<T>`.

<https://wg21.link/p3074r2>